School of Future Tech

Case Study Report

On

Restaurant Order & Kitchen Management System

By

Manan Jain

150096725166

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Running a restaurant involves a lot of moving parts, especially when it comes to making sure the right food gets to the right table on time. In many traditional setups, orders are written on paper, which can lead to mistakes, lost tickets, or confusion between the servers and the kitchen staff.

This case study focuses on creating a Restaurant Order & Kitchen Management System using C++. The goal is to digitize the flow of information. Instead of manual notes, the system handles everything from taking the order and displaying it in the kitchen to generating the final bill with taxes. It's designed to be a simple, central way to manage the daily operations of a small eatery or cafe.

2. Problem Statement / Case Background (Abstract)

Background

The biggest issue in manual restaurant management is the delay in communication. When a server takes an order, they have to physically walk it to the kitchen. If the customer wants to change something or cancel an item, it creates more back-and-forth. Additionally, calculating the final bill manually with GST can lead to human error.

Abstract

This project provides a console-based solution to automate these tasks. Built using C++ the system allows staff to input orders for specific tables. These orders are then sent to a virtual kitchen display where chefs can update the status (e.g from Pending to Preparing). The system also handles item cancellations and automatically calculates the subtotal and

GST for the final bill. By using parallel arrays to store data, the system keeps track of multiple items per order ensuring that the kitchen and the billing counter stay perfectly synced.

3. Case Study Design

The system is designed as a linear flow to mimic how a real restaurant operates:

1. **Menu Management:** A fixed list of items and prices is stored so that staff only need to enter item numbers.
2. **Order Entry:** Staff can select multiple items for a single table. Each item is assigned an Order ID to keep the group together.
3. **Kitchen Workflow:** A dedicated display shows only the items that need cooking. The status of each item can be updated in real-time.
4. **Billing & Taxes:** Once the kitchen marks an order as Ready the system pulls the data, applies a 5% GST and prints a formatted receipt.
5. **Data Persistence (Current Session):** All active orders are stored in arrays during the program's execution for quick viewing and updates.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Logic & Methods:

- **Parallel Array Storage:** Since we are using basic C++, I used several arrays (like orderId[ ], orderStatus[ ], and orderAmount[ ]) that work together. Each index across these arrays represents one part of a single order.
- **State Management:** I used string-based statuses (Pending, Preparing, Ready Billed) to control the flow of the program. For example, an item cannot be billed unless its status is Ready.
- **Input Validation:** The code checks if an item exists (1-6) or if the quantity is valid before saving it to the system.

Algorithms used:

1. **Order Processing:**
    1. Assign a unique nextOrderId to the current table.
    2. Loop through item selection until the user says no.
    3. Calculate the price for each line item (Price × Qty) and store it.
2. **Kitchen Update:**
    1. Search the orderStatus array for Pending items.
    2. Filter results by the user-provided Order ID.
    3. Update the status string based on the chef's input.
3. **Billing Logic:**
    1. Search for all items matching a specific Order ID that are Ready. Sum the orderAmount for these items.
    2. Calculate GST (5%) and add it to the subtotal for the final amount.

Technology Stack:

- **Language:** C++
- **Standard Libraries:** <iostream> for input/output and <string> for handling text-based statuses and menu names.
- **Tools:** Any standard C++ compiler (G++, VS Code, or Dev-C++).

5. Problem Statement / Case Study Implementation Details and Snapshots.

The code is organized into several functional blocks:

- takeOrder(): Handles the main data entry for new customers.
- kitchenDisplay(): Acts as the interface for the back-of-house staff.
- generateBill(): Performs the final calculations and "closes" the order.
- cancelItem(): Provides a way to remove specific lines from an order if a mistake was made.

file.cpp > takeOrder()

```cpp
#include <iostream>
#include <string>
using namespace std;
string menuName[6]  = {"Burger", "Pizza", "Cold Coffee", "Lemon Soda", "Vanilla Ice Cream", "Chocolate Ice Cream"};
float  menuPrice[6] = {80, 150, 60, 40, 50, 60};
int    orderId[50], orderTable[50], orderItem[50], orderQty[50];
float  orderAmount[50];
string orderStatus[50];
int    totalItems = 0, nextOrderId = 1;

void showMenu() {
    cout << "\n--- Menu ---" << endl;
    for (int i = 0; i < 6; i++)
        cout << i+1 << ". " << menuName[i] << "  -  Rs." << menuPrice[i] << endl;
    cout << "------------" << endl;
}

void takeOrder() {
    int table, itemNo, qty;
    char more = 'y';
    int id = nextOrderId++;

    cout << "\nTable number : "; cin >> table;
    showMenu();

    while (more == 'y' || more == 'Y') {
        cout << "Which item? (1-6) : "; cin >> itemNo;
        if (itemNo < 1 || itemNo > 6) { cout << "That item doesn't exist, try again." << endl; continue; }

        cout << "How many? : "; cin >> qty;
        if (qty <= 0) { cout << "Quantity can't be zero or less." << endl; continue; }

        orderId[totalItems]     = id;
        orderTable[totalItems]  = table;
        orderItem[totalItems]   = itemNo - 1;
        orderQty[totalItems]    = qty;
        orderAmount[totalItems] = menuPrice[itemNo-1] * qty;
        orderStatus[totalItems] = "Pending";
        totalItems++;

        cout << menuName[itemNo-1] << " x" << qty << " added." << endl;
        cout << "Anything else? (y/n) : "; cin >> more;
    }
    cout << "Order " << id << " placed for table " << table << ". Status : Pending" << endl;
}

void viewOrders() {
    if (totalItems == 0) { cout << "\nNo orders placed yet." << endl; return; }
    cout << "\n--- All Orders ---" << endl;
    int prevId = -1;
    for (int i = 0; i < totalItems; i++) {
        if (orderId[i] != prevId) {
            cout << "\nOrder " << orderId[i] << " | Table " << orderTable[i] << " | Status : " << orderStatus[i] << endl;
            prevId = orderId[i];
        }
        cout << "  [Line " << i+1 << "] " << menuName[orderItem[i]]
             << " x" << orderQty[i] << " = Rs." << orderAmount[i]
```

Ln 32, Col 1 Spaces: 4 UT

```cpp
void viewOrders() {
    if (totalItems == 0) { cout << "\nNo orders placed yet." << endl; return; }
    cout << "\n--- All Orders ---" << endl;
    int prevId = -1;
    for (int i = 0; i < totalItems; i++) {
        if (orderId[i] != prevId) {
            cout << "\nOrder " << orderId[i] << " | Table " << orderTable[i] << " | Status : " << orderStatus[i] << endl;
            prevId = orderId[i];
        }
        cout << "  [Line " << i+1 << "] " << menuName[orderItem[i]]
             << " x" << orderQty[i] << " = Rs." << orderAmount[i]
             << " (" << orderStatus[i] << ")" << endl;
    }
    cout << "-----------------" << endl;
}

void kitchenDisplay() {
    cout << "\n--- Kitchen ---" << endl;
    bool found = false;
    int prevId = -1;
    for (int i = 0; i < totalItems; i++) {
        if (orderStatus[i] != "Pending" && orderStatus[i] != "Preparing") continue;
        if (orderId[i] != prevId) { cout << "\nOrder " << orderId[i] << " | Table " << orderTable[i] << endl; prevId = orderId[i]; }
        cout << "  -> " << menuName[orderItem[i]] << " x" << orderQty[i] << " (" << orderStatus[i] << ")" << endl;
        found = true;
    }
    if (!found) { cout << "Kitchen is clear, no pending orders right now." << endl; return; }

    int id, choice;
    cout << "\nEnter order ID to update (0 to go back) : "; cin >> id;
    if (id == 0) return;
    cout << "1. Mark as Preparing   2. Mark as Ready : "; cin >> choice;

    if (choice == 1 || choice == 2) {
        string s = (choice == 1) ? "Preparing" : "Ready";
        for (int i = 0; i < totalItems; i++)
            if (orderId[i] == id) orderStatus[i] = s;
        cout << "Order " << id << " updated to " << s << "." << endl;
    }
    else cout << "Invalid choice." << endl;
}

void cancelItem() {
    cout << "\n--- Cancel an Item ---" << endl;
    int prevId = -1;
    for (int i = 0; i < totalItems; i++) {
        if (orderStatus[i] == "Cancelled" || orderStatus[i] == "Billed") continue;
        if (orderId[i] != prevId) { cout << "\nOrder " << orderId[i] << " | Table " << orderTable[i] << endl; prevId = orderId[i]; }
        cout << "  [Line " << i+1 << "] " << menuName[orderItem[i]] << " x" << orderQty[i] << " = Rs." << orderAmount[i] << endl;
    }

    int lineNo;
    cout << "\nEnter line number to cancel (0 to go back) : "; cin >> lineNo;
    if (lineNo == 0) return;
    lineNo--;

```

```cpp

    if (lineNo < 0 || lineNo >= totalItems) { cout << "That line number doesn't exist." << endl; return; }
    if (orderStatus[lineNo] == "Billed" || orderStatus[lineNo] == "Cancelled") { cout << "Can't cancel this item anymore." << endl; return; }

    orderStatus[lineNo] = "Cancelled";
    cout << menuName[orderItem[lineNo]] << " has been removed from the order." << endl;
}

void generateBill() {
    cout << "\n--- Bill ---" << endl;
    bool anyReady = false;
    int prevId = -1;
    for (int i = 0; i < totalItems; i++) {
        if (orderStatus[i] == "Ready" && orderId[i] != prevId) {
            cout << "Order " << orderId[i] << " | Table " << orderTable[i] << endl;
            prevId = orderId[i]; anyReady = true;
        }
    }
    if (!anyReady) { cout << "No orders are ready for billing yet." << endl; return; }

    int id;
    cout << "Enter order ID to bill : "; cin >> id;

    float subtotal = 0; bool found = false;
    for (int i = 0; i < totalItems; i++)
        if (orderId[i] == id && orderStatus[i] == "Ready") { subtotal += orderAmount[i]; found = true; }

    if (!found) { cout << "Order not found or not ready yet." << endl; return; }

    float tax = subtotal * 0.05;
    cout << "\n---- Bill for Order " << id << " ----" << endl;
    for (int i = 0; i < totalItems; i++)
        if (orderId[i] == id && orderStatus[i] == "Ready")
            cout << menuName[orderItem[i]] << " x" << orderQty[i] << " = Rs." << orderAmount[i] << endl;
    cout << "-----------------------------" << endl;
    cout << "Subtotal : Rs." << subtotal << endl;
    cout << "GST (5%) : Rs." << subtotal * 0.05 << endl;
    cout << "Total    : Rs." << subtotal + tax << endl;
    cout << "-----------------------------" << endl;
    cout << "Thanks for dining with us, see you again!" << endl;

    for (int i = 0; i < totalItems; i++)
        if (orderId[i] == id) orderStatus[i] = "Billed";
}

int main() {
    cout << "================================" << endl;
    cout << "  Restaurant Management System"  << endl;
    cout << "================================" << endl;

    int choice;
    do {
        cout << "\n1. Show Menu"     << endl;
        cout << "2. Take Order"      << endl;
        cout << "3. Kitchen Display" << endl;
```

```cpp
    cout << "-----------------------------" << endl;
    cout << "Thanks for dining with us, see you again!" << endl;

    for (int i = 0; i < totalItems; i++)
        if (orderId[i] == id) orderStatus[i] = "Billed";
}

int main() {
    cout << "==============================" << endl;
    cout << "  Restaurant Management System"  << endl;
    cout << "==============================" << endl;

    int choice;
    do {
        cout << "\n1. Show Menu"     << endl;
        cout << "2. Take Order"      << endl;
        cout << "3. Kitchen Display" << endl;
        cout << "4. Generate Bill"   << endl;
        cout << "5. View All Orders" << endl;
        cout << "6. Cancel Item"     << endl;
        cout << "7. Exit"            << endl;
        cout << "\nWhat would you like to do? "; cin >> choice;

        if      (choice == 1) showMenu();
        else if (choice == 2) takeOrder();
        else if (choice == 3) kitchenDisplay();
        else if (choice == 4) generateBill();
        else if (choice == 5) viewOrders();
        else if (choice == 6) cancelItem();
        else if (choice == 7) cout << "Closing system. Goodbye!" << endl;
        else                  cout << "Please enter a number between 1 and 7." << endl;

    } while (choice != 7);
    return 0;
}
```

OutPut

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mananjain@Manans-MacBook-Air Major-Project ( C++) % cd "/Users/mananjain/Documents/Major-Project ( C++)/" && g++ file.cpp -o file && "/Users/mananjain/Documents/Major-Project ( C++)/"file
------------
Which item? (1-6) : 2
How many? : 2
Pizza x2 added.
Anything else? (y/n) : y
Which item? (1-6) : 6
How many? : 1
Chocolate Ice Cream x1 added.
Anything else? (y/n) : n
Order 1 placed for table 1. Status : Pending

1. Show Menu
2. Take Order
3. Kitchen Display
4. Generate Bill
5. View All Orders
6. Cancel Item
7. Exit

What would you like to do? 3

--- Kitchen ---

Order 1 | Table 1
  -> Pizza x2 (Pending)
  -> Chocolate Ice Cream x1 (Pending)

Enter order ID to update (0 to go back) : 1
1. Mark as Preparing   2. Mark as Ready : 2
Order 1 updated to Ready.

1. Show Menu
2. Take Order
3. Kitchen Display
4. Generate Bill
5. View All Orders
6. Cancel Item
7. Exit

What would you like to do? 4

--- Bill ---
Order 1 | Table 1
Enter order ID to bill : 1

---- Bill for Order 1 ----
Pizza x2 = Rs.300
Chocolate Ice Cream x1 = Rs.60
-----------------------------
Subtotal : Rs.360
GST (5%) : Rs.18
Total    : Rs.378
-----------------------------
Thanks for dining with us, see you again!

1. Show Menu
2. Take Order
3. Kitchen Display
4. Generate Bill
5. View All Orders
6. Cancel Item
7. Exit
                                                    Ln 155, Col 1    Spaces: 4    U
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- Using Parallel Arrays is an effective way to manage related data without needing complex database structures for a simple system.
- The Status-based logic (Pending -> Ready -> Billed) prevents errors, such as accidentally billing for food that hasn't been prepared yet.
- Automating the Tax Calculation ensures that every bill is mathematically correct, which is a big improvement over manual calculation.

Conclusion

This project successfully automates the basic needs of a restaurant. It bridges the gap between the dining area and the kitchen, making sure that orders are tracked from the moment they are placed until the bill is paid. Building this system helped me understand how to manage data flow and how to use loops and conditional logic to solve a real-world coordination problem.

7. References

- C++ Programming Language Documentation (cplusplus.com) for array and string handling.
- Standard I/O tutorials for console-based formatting.
- Introduction to Programming Logic - College Course Notes.